

The Mathematics of Vikasa

This document explains only the mathematics used by the executable Vikasa simulator. It describes the implementation at commit 0140311 (the current v1.1 engine), not the proposed Vikasa v2 design. Every formula below is derived from the source code in `src/evolution_sim`.

1. Mathematical notation

Symbol	Meaning
t	current simulation tick
i,j	organism or gene indexes
$x=(x,y)$	a two-dimensional position
$v=(v_x,v_y)$	a two-dimensional velocity
$ v _2$	Euclidean length of a vector
$U(a,b)$	continuous uniform random value from $[a,b]$
$N(\mu,\sigma)$	normally distributed value with mean μ and standard deviation σ
$clip(x,a,b)$	restrict x to the closed interval $[a,b]$
$\lfloor x \rfloor$	floor: the greatest integer no larger than x
$\bar{x}$	arithmetic mean

One tick is the engine's atomic unit. In the current implementation it is not assigned a real-world number of days or years. The GUI speed choices are exactly 1, 2, 4, 8, or 16 simulation ticks per rendered frame.

2. The six-dimensional genome

Each organism has a genome vector

$$\mathbf{g} = (g_s, g_v, g_p, g_m, g_r, g_f),$$

where the coordinates are size, speed, perception, metabolism, reproduction threshold, and fertility. The shipped default bounds are:

Trait	Symbol	Minimum	Maximum
Size	g_s	2.5	8.0
Speed	g_v	0.5	4.0
Perception	g_p	25.0	150.0
Metabolism	g_m	0.65	1.5
Reproduction threshold	g_r	70.0	150.0
Fertility	g_f	0.15	0.55

Every coordinate must be finite and inside its configured interval.

2.1 Initial genomes: independent uniform distributions

For gene i , with configured minimum L_i and maximum H_i , initial organisms receive

$$g_i \sim U(L_i, H_i).$$

The six draws are independent conditional on the pseudo-random generator state. This is a rectangular distribution over the permitted six-dimensional genome space; it is not a normal distribution around an ideal organism.

Relevant mathematical idea: a continuous uniform distribution gives equal probability density to every value in its interval.

2.2 Clamping

Whenever mutation could move a gene outside its legal interval, Vikasa applies

$$\text{clip}(x, L, H) = \min(\max(x, L), H).$$

Clamping preserves the invariant $L \leq g_i \leq H$. It also creates extra probability mass at the boundaries when a mutation overshoots.

3. Inheritance and genetic variation

Two parent genomes a and b create an intermediate child genome by one of two crossover rules.

3.1 Uniform crossover

For every gene i , independently draw $u_i \sim U(0, 1)$:

$c_i =$

$a_i, u_i < 0.5,$

$b_i, u_i \geq 0.5.$

Each child gene therefore comes exactly from one parent, with probability $1/2$ for each.

Relevant mathematical idea: this is a Bernoulli trial with success probability $p=0.5$.

3.2 Arithmetic crossover

The default configuration uses arithmetic crossover. For each gene, draw $\alpha_i \sim U(0, 1)$ and compute

$$c_i = \alpha_i a_i + (1 - \alpha_i) b_i.$$

This is a convex combination. Because $0 \leq \alpha_i \leq 1$, the result lies between the two parental values:

$$\min(a_i, b_i) \leq c_i \leq \max(a_i, b_i).$$

That interval property follows directly from the convex-combination theorem.

3.3 Mutation probability

For each gene, Vikasa independently draws $u_i \sim U(0, 1)$. Mutation occurs when

$$u_i < p_m,$$

where the default mutation probability is $p_m=0.08$, and the showcase configuration uses $p_m=0.10$.

With six independent genes, the number of mutated genes K follows a binomial distribution:

$$K \sim \text{Binomial}(6, p_m).$$

Therefore,

$$P(K = k) = \binom{6}{k} p_m^k (1 - p_m)^{6-k}, \quad E[K] = 6p_m.$$

At the default $p_m=0.08$, the expected mutations per newborn are 0.48 , and the probability of no gene mutating is

$$(1 - 0.08)^6 \approx 0.6064.$$

3.4 Mutation magnitude

Let the legal span of gene i be

$$S_i = H_i - L_i.$$

For a selected gene, the mutation is

$$\Delta_i = S_i Z_i, \quad Z_i \sim N(0, \sigma_m),$$

and the final child value is

$$g'_i = \text{clip}(c_i + \Delta_i, L_i, H_i).$$

The default mutation sigma is $\sigma_m=0.06$. For size, whose span is $8.0-2.5=5.5$, the pre-clamp mutation standard deviation is

$$5.5(0.06) = 0.33.$$

The Gaussian is centered at zero, so upward and downward mutations are equally likely before clamping.

4. Initial physical state

4.1 Position

In a world of width W and height H , initial positions and newly spawned food positions are

$$x \sim U(0, W), \quad y \sim U(0, H).$$

The default world is 1200×760 .

4.2 Direction and velocity

An initial heading is

$$\theta \sim U(0, 2\pi).$$

The initial speed magnitude is a fraction of the inherited speed gene:

$$q = g_v U(0.15, 0.45).$$

The velocity vector is

$$\mathbf{v} = q(\cos \theta, \sin \theta).$$

This is the polar-to-Cartesian coordinate transformation.

4.3 Initial age

If the minimum reproductive age is A_{min} , the initial age is drawn as a discrete uniform integer

$$A \in \{0, 1, \dots, A_{min} - 1\}.$$

This prevents every founder from becoming reproductive on the same tick.

5. Vector mathematics

5.1 Euclidean norm

For $v=(v_x, v_y)$, Vikasa uses

$$\|\mathbf{v}\|_2 = \sqrt{v_x^2 + v_y^2}.$$

This is the Pythagorean theorem applied to Cartesian coordinates.

5.2 Unit vector

For a nonzero vector,

$$\hat{\mathbf{v}} = \frac{\mathbf{v}}{\|\mathbf{v}\|_2}.$$

For the zero vector, Vikasa returns $(0,0)$ rather than dividing by zero.

5.3 Squared distance

The squared Euclidean distance between positions a and b is

$$d^2(\mathbf{a}, \mathbf{b}) = (a_x - b_x)^2 + (a_y - b_y)^2 = (\mathbf{a} - \mathbf{b}) \cdot (\mathbf{a} - \mathbf{b}).$$

Most comparisons use d^2 instead of d . Because the square-root function is strictly increasing on nonnegative numbers,

$$d_1 < d_2 \iff d_1^2 < d_2^2,$$

so removing the square root preserves ordering while saving work.

6. Finding food and moving

6.1 Perception disk

An organism detects a food resource when

$$d^2(x_{\text{organism}}, x_{\text{food}}) \leq g_p^2.$$

Among visible resources, Vikasa chooses the smallest ordered pair

$$(d^2, \text{resource_id}).$$

Thus the nearest food wins; an equal-distance tie is resolved by the smaller stable resource ID.

6.2 Wandering

If no food is visible, direction is generated from the organism's persistent wander angle θ . With configured probability p_w per tick,

$$\theta \leftarrow \theta + \varepsilon, \quad \varepsilon \sim N(0, 0.65).$$

Then

$$\mathbf{d} = (\cos \theta, \sin \theta).$$

The default wander-change probability is $p_w=0.025$, so the expected waiting time between changes is $1/p_w=40$ ticks. This uses the mean of a geometric distribution.

6.3 Steering interpolation

The desired velocity is

$$\mathbf{v}_{\text{desired}} = g_v \mathbf{d}.$$

The current velocity is moved 30% toward the desired velocity:

$$\mathbf{v}_{\text{new}} = \mathbf{v}_{\text{old}} + 0.3(\mathbf{v}_{\text{desired}} - \mathbf{v}_{\text{old}}) = 0.7\mathbf{v}_{\text{old}} + 0.3\mathbf{v}_{\text{desired}}.$$

This is linear interpolation, also called exponential smoothing when repeated over time. If $|\mathbf{v}_{\text{new}}|_2 > g_v$, it is capped:

$$\mathbf{v}_{new} \leftarrow g_v \frac{\mathbf{v}_{new}}{\|\mathbf{v}_{new}\|_2}.$$

Position then advances by one velocity vector:

$$\mathbf{x}_{t+1} = \mathbf{x}_t + \mathbf{v}_{new}.$$

The distance used for movement energy is $\|\mathbf{v}_{new}\|_2$.

7. World boundaries

7.1 Collision/reflection boundary

The default world clamps a coordinate to its boundary and reverses the outward velocity component.

For the left edge:

$$x < 0 \Rightarrow x = 0, \quad v_x = |v_x|.$$

For the right edge:

$$x > W \Rightarrow x = W, \quad v_x = -|v_x|.$$

The same rule applies at $y=0$ and $y=H$. This is an axis-aligned reflection model; overshoot distance is discarded rather than reflected repeatedly.

7.2 Wrap boundary

When the configuration selects wrap, Vikasa uses modular arithmetic:

$$x \mapsto x \bmod W,$$

$$y \mapsto y \bmod H.$$

This makes the rectangle topologically similar to a torus for boundary crossing, although distance queries themselves are not toroidal.

8. Energy calculations

Let:

- B be configured basal cost;
- M be configured movement cost;
- $s=g_s$ be size;
- $m=g_m$ be metabolism;
- $v_{max}=g_v$ be the speed gene;
- $d=\|\mathbf{v}\|_2$ be distance moved this tick;
- E_m be the current environmental metabolic multiplier.

8.1 Basal energy cost

$$C_{basal} = B(1 + \frac{s}{8})^{\frac{1}{m}} E_m.$$

Larger organisms pay more. In this model a larger metabolism gene reduces basal cost because the formula divides by m . A heat event can multiply the result through E_m .

8.2 Movement energy cost

$$C_{move} = Md(0.5 + \frac{s}{8})(0.5 + \frac{v_{max}}{4}).$$

Movement cost grows linearly with actual distance, linearly with a size factor, and linearly with a speed-capability factor.

8.3 Energy recurrence

Before feeding,

$$E_{t+1} = E_t - C_{\text{basal}} - C_{\text{move}}.$$

Worked example using default values $B=0.025$, $M=0.018$, $s=5$, $m=1$, $v_{\text{max}}=2$, $d=1.5$, and $E_m=1$:

$$C_{\text{basal}} = 0.025(1 + 5/8) = 0.040625,$$

$$C_{\text{move}} = 0.018(1.5)(0.5 + 5/8)(0.5 + 2/4) = 0.030375.$$

Total cost is 0.071 energy units for that tick.

9. Food regeneration and consumption

9.1 Fractional spawn accumulator

Let r be configured resources per tick and F_t the environment's food multiplier. Vikasa maintains a fractional accumulator a_t :

$$a'_{t+1} = a_t + rF_t,$$

$$n_t = \lfloor a'_{t+1} \rfloor,$$

$$a_{t+1} = a'_{t+1} - n_t.$$

It attempts to spawn n_t resources, limited by the maximum resource count. This preserves fractional rates exactly over time. For example, $r=0.25$ creates one resource every four ticks when the multiplier is 1.

9.2 Collision with food

Treat an organism and resource as disks. If organism size is s and resource radius is r_f , consumption is possible when

$$d^2 \leq (s+r_f)^2.$$

This is the standard circle-intersection condition.

If multiple organisms overlap one food item, the winner minimizes

$$(d^2, \text{organism_id}).$$

9.3 Feeding and saturation

For resource energy Q and maximum energy E_{max} :

$$E_{\text{new}} = \min(E_{\text{max}}, E_{\text{old}} + Q).$$

The analytical food_acquired counter always receives the full Q , even if the organism's stored energy saturates below $E_{\text{old}}+Q$.

10. Reproduction

10.1 Fertility-adjusted cooldown

For configured base cooldown C and fertility gene $f=g_f$, the required cooldown is

$$C_{\text{effective}} = \max(1, \text{round}(C(1 - 0.5f))).$$

At the default $C=160$ and $f=0.4$:

$$C_{\text{effective}} = \text{round}(160(1 - 0.2)) = 128.$$

10.2 Eligibility predicate

An organism may reproduce exactly when all three inequalities hold:

$$\begin{aligned} \text{age} &\geq A_{\min}, \\ \text{energy} &\geq g_r, \\ t - t_{\text{last}} &\geq C_{\text{effective}}. \end{aligned}$$

There is no separate random probability of reproduction after eligibility.

10.3 Pair formation

Eligible organisms are processed by increasing stable ID. A partner must be within mate radius R_m :

$$d^2 \leq R_m^2.$$

The nearest eligible, unpaired partner is selected, with smaller ID breaking distance ties. Each organism can participate in at most one pair per tick.

10.4 Child position

For parent positions x_1 and x_2 , first compute the midpoint

$$\mathbf{m} = \frac{\mathbf{x}_1 + \mathbf{x}_2}{2}.$$

Then add independent Gaussian displacement to each coordinate:

$$\mathbf{x}_{\text{child}} = \text{clip}(\mathbf{m} + (\varepsilon_x, \varepsilon_y), (0, 0), (W, H)),$$

where $\varepsilon_x, \varepsilon_y \sim N(0, 2)$. The child begins with zero velocity and a new random wander angle.

10.5 Reproductive energy transfer

For configured offspring energy E_o , the child receives E_o , and each parent pays

$$\frac{E_o}{2}.$$

The total transfer is therefore energy-conserving at the reproduction step:

$$-\frac{E_o}{2} - \frac{E_o}{2} + E_o = 0.$$

This does not include subsequent metabolism or any clipping effect elsewhere.

10.6 Population cap

Birth is suppressed when

$$N_{\text{living}} + N_{\text{pending}} \geq N_{\text{cap}}.$$

The default cap is 800; the showcase cap is 600.

11. Death

An organism is removed when either

$$E \leq 0$$

or

$$age > A_{max}.$$

Notice the strict age comparison: an organism remains alive at exactly A_{max} and dies after exceeding it.

12. Environmental multipliers

An event with start s and duration D is active on the half-open tick interval

$$s \leq t < s + D.$$

Every tick begins with neutral multipliers

$$F_t = 1, \quad E_m = 1.$$

For active drought or abundance events with intensity I_k , food effects multiply:

$$F_t = \prod_{k \in \mathcal{F}_t} I_k.$$

For active heat events, metabolic effects multiply:

$$E_m = \prod_{k \in \mathcal{H}_t} I_k.$$

Multiplication makes overlapping proportional effects compound. Two food events with intensities 0.5 and 0.8 produce $F_t = 0.4$, not 0.3 or 1.3.

Current event names are drought, abundance, and heat. More extensive climate mathematics appears in the approved v2 design but is not yet executable and is intentionally excluded here.

13. Spatial hashing

The simulator avoids checking every entity against every other entity by dividing space into square cells of width h .

13.1 Cell coordinates

Position (x, y) belongs to cell

$$c_x = \lfloor \frac{x}{h} \rfloor, \quad c_y = \lfloor \frac{y}{h} \rfloor.$$

The resource index chooses

$$h = \max(24, \frac{P_{max}}{2}),$$

where P_{max} is the configured maximum perception value. The mating index uses $h = \max(1, R_m)$.

13.2 Radius query

For query center x and radius R , the algorithm first visits cells from

$$cell(\mathbf{x} - R) \text{ through } cell(\mathbf{x} + R).$$

Each candidate then passes the exact disk test

$$d^2 \leq R^2.$$

Results are sorted by stable integer ID. The grid reduces expected local-query work compared with an all-pairs search, while the final distance test prevents square-cell false positives.

14. Determinism and pseudo-randomness

The engine creates one NumPy pseudo-random generator from an integer seed:

`RNG = default_rng(seed).`

All random positions, genomes, headings, wander changes, crossover weights, mutations, and child offsets consume this generator in a fixed order. Organisms and contested resources are processed by sorted IDs. Therefore identical configuration, seed, and action sequence produce identical snapshots.

This is deterministic pseudo-randomness, not true randomness. The seed selects one reproducible sequence from the generator.

15. Population statistics

Suppose one trait has living values $x_1, \dots, x_n$.

15.1 Arithmetic mean

$$\bar{x} = \frac{1}{n} \sum_{i=1}^n x_i.$$

15.2 Median

After sorting the values, the median is the middle value for odd n , or the mean of the two middle values for even n .

15.3 Population variance and standard deviation

Vikasa uses NumPy's population variance (`ddof=0`):

$$\sigma_x^2 = \frac{1}{n} \sum_{i=1}^n (x_i - \bar{x})^2.$$

The population standard deviation is

$$\sigma_x = \sqrt{\sigma_x^2}.$$

These are descriptive statistics of the complete living population at that tick, not unbiased estimators of an unseen population.

15.4 Pearson trait–offspring correlation

Let x_i be a trait and y_i the offspring count of organism i . Vikasa reports the Pearson correlation

$$r_{xy} = \frac{\sum_i (x_i - \bar{x})(y_i - \bar{y})}{\sqrt{\sum_i (x_i - \bar{x})^2} \sqrt{\sum_i (y_i - \bar{y})^2}}.$$

It returns no correlation when $n < 2$, when the trait has zero variance, or when offspring counts have zero variance, because the denominator would be zero. Valid results are rounded to 12 decimal places.

Correlation measures linear association; it does not prove that a trait caused reproductive success.

15.5 Analytical fitness score

For display and export only, the simulator calculates

$$fitness = 100O + 0.01A + 0.5Q,$$

where O is offspring count, A is age in ticks, and Q is lifetime food energy acquired. The engine never uses this score to decide survival or reproduction. It is a reporting index, not a theorem-derived biological fitness measure.

Mean analytical fitness is the arithmetic mean of this score across living organisms.

16. Genetic diversity

16.1 Normalizing genes

Because traits have different units and ranges, each gene is normalized:

$$z_i = \frac{g_i - L_i}{H_i - L_i}.$$

Every normalized coordinate lies in $[0, 1]$.

16.2 Pairwise genetic distance

For normalized genomes $z^{(a)}$ and $z^{(b)}$:

$$d_{ab} = \|z^{(a)} - z^{(b)}\|_2 = \sqrt{\sum_{i=1}^6 (z_i^{(a)} - z_i^{(b)})^2}.$$

The diversity statistic is the mean over all unordered pairs:

$$D = \frac{1}{\binom{n}{2}} \sum_{a < b} d_{ab}.$$

For six normalized genes,

$$0 \leq D \leq \sqrt{6}.$$

If fewer than two organisms live, $D=0$.

16.3 Deterministic subsampling

Pairwise distance requires $\binom{n}{2} = n(n-1)/2$ comparisons. When population exceeds the configured diversity sample limit m (default 128), Vikasa selects m evenly spaced indexes from 0 through $n-1$. It then computes diversity on those organisms. This bounds pair comparisons at

$$\binom{128}{2} = 8128.$$

The sample is deterministic rather than random, preserving reproducibility.

17. Moving averages

For measurements $x_0, x_1, \dots$ and positive window w , the reported trailing average at index t is

$$MA_t = \frac{1}{k_t} \sum_{j=\max(0, t-w+1)}^t x_j,$$

where

$$k_t = t - \max(0, t - w + 1) + 1.$$

At the beginning of a series, Vikasa uses all available values instead of waiting for a complete window.

18. Lineage graph mathematics

Parent-child records form a directed graph. Each child has exactly two parent IDs, while a parent may have any number of children.

Ancestor and descendant queries use breadth-first search. Starting depth is zero. Neighbors are added at depth $d+1$. With maximum depth k , traversal stops expanding nodes when

$$d \geq k.$$

The visited set prevents repeated work when the same ancestor is reachable through more than one path.

19. Rendering mathematics

Rendering never feeds values back into the simulation.

19.1 World-to-screen projection

For world position (x,y) , world dimensions (W,H) , and viewport rectangle with origin (L,T) , width W_s , and height H_s :

$$x_s = \text{round} \left(L + \frac{x}{W} W_s \right),$$

$$y_s = \text{round} \left(T + \frac{y}{H} H_s \right).$$

This is independent linear scaling on the two axes.

19.2 Organism radius and hit testing

The displayed body radius is

$$r_{\text{body}} = \max(4, \text{round}(1.12g_s)).$$

Mouse hit testing deliberately uses the larger radius

$$r_{\text{hit}} = \max(7, 1.25g_s).$$

A click selects a body when

$$(x_{\text{click}} - x_s)^2 + (y_{\text{click}} - y_s)^2 \leq r_{\text{hit}}^2.$$

If bodies overlap, the closest screen center wins, with stable organism ID breaking a tie.

19.3 Trait-to-color normalization

Speed becomes a color interpolation amount:

$$h = \frac{g_v - v_{\min}}{v_{\max} - v_{\min}}.$$

For two RGB colors C_1 and C_2 , color mixing uses

$$C(h) = \text{round} (C_1 + h(C_2 - C_1)),$$

with h clamped to $[0,1]$. This is component-wise linear interpolation.

19.4 Energy color

Energy is normalized as

$$e = \text{clip} \left(\frac{E}{E_{\max}}, 0, 1 \right).$$

The inner body color interpolates from the pressure color at $e=0$ to the organism's trait color at $e=1$.

19.5 Heading marker

Velocity direction is recovered with

$$\theta = \text{atan2}(v_y, v_x).$$

The nose point is

$$(x_n, y_n) = (\text{round}(x_s + (r + 3)\cos\theta), \text{round}(y_s + (r + 3)\sin\theta)).$$

19.6 Perception circle on screen

The displayed perception radius uses the horizontal scale:

$$r_{\text{screen}} = \text{round}\left(\frac{g_p}{W}W_s\right).$$

20. Chart normalization

For a series $y_0, \dots, y_{n-1}$ drawn inside a rectangle of width W_c , height H_c , left L , and bottom B , define

$$y_{\min} = \min_i y_i, \quad y_{\max} = \max_i y_i,$$

$$S = \max(10^{-9}, y_{\max} - y_{\min}).$$

Each chart point is

$$x_i = \text{round}\left(L + \frac{iW_c}{\max(1, n-1)}\right),$$

$$y_i^{\text{screen}} = \text{round}\left(B - \frac{y_i - y_{\min}}{S}H_c\right).$$

The 10^{-9} lower bound prevents division by zero for a constant series.

Static trait charts normalize a mean trait value μ_i with the same min-max equation used by diversity:

$$\mu_i^{\text{normalized}} = \frac{\mu_i - L_i}{H_i - L_i}.$$

21. GUI layout calculations

The window is first clamped to at least 1180×720 . For window width W_u and height H_u :

$$\text{sidebar_width} = \max(276, \min(330, \text{round}(0.225W_u))),$$

$$\text{chart_height} = \max(154, \min(204, \text{round}(0.205H_u))).$$

The remaining rectangles are formed from fixed 16-pixel margins, 12-pixel gaps, and a 68-pixel top bar. These equations keep the main world, sidebar, and chart strip non-overlapping.

22. What is a theorem and what is a model assumption?

The simulator combines standard mathematical results with chosen artificial-life rules.

Standard mathematical results used

- Pythagorean theorem: Euclidean norm and distance.
- Monotonicity of square root: squared distances preserve nearest-neighbor ordering.
- Convex-combination interval property: arithmetic-crossover genes remain between parental values before mutation.
- Bernoulli and binomial distributions: independent mutation decisions.
- Gaussian distribution: mutation, wandering, and newborn displacement.
- Linear interpolation: steering, colors, and coordinate projection.
- Modular arithmetic: wrap boundaries.
- Circle-intersection inequality: food contact and radius queries.
- Arithmetic mean, population variance, standard deviation, median, and Pearson correlation.
- Breadth-first graph traversal: ancestors and descendants.

Deliberate model assumptions

- The six gene meanings and their allowed ranges.
- Dividing basal cost by the metabolism gene.
- The constants 0.3, 8, 4, 0.5, 100, 0.01, and 0.5 in steering, energy, cooldown, and reporting fitness.
- Nearest-resource movement and nearest-eligible mating.
- Exactly two parents and at most one mating per organism per tick.
- Death at nonpositive energy or age above the configured maximum.
- Multiplicative environment effects.
- The analytical fitness weights.

These assumptions are not theorems. They define Vikasa's simulated world and can be changed through code or, where exposed, configuration.

23. Source map

Mathematics	Implementation
Genome bounds and initial sampling	src/evolution_sim/model/genome.py
Crossover and mutation	src/evolution_sim/model/genetics.py
Norm, squared distance, reflection	src/evolution_sim/model/math2d.py
Spatial hashing	src/evolution_sim/model/spatial.py
Movement, energy, food, reproduction, death	src/evolution_sim/simulation/engine.py
Environmental products and active intervals	src/evolution_sim/simulation/environment.py
Population statistics, fitness, diversity	src/evolution_sim/analytics/metrics.py
World projection and organism appearance	src/evolution_sim/ui/renderer.py
Live chart mapping	src/evolution_sim/ui/charts.py
Static chart normalization	src/evolution_sim/experiments/charts.py
Lineage graph traversal	src/evolution_sim/model/lineage.py
Numeric constants and legal ranges	config/default.json, config/showcase.json

This source map is the authority if the implementation changes after this document's recorded commit.